

Lab 2 Report: Written Questions

Q1: When you publish a message or call a service, why does only your turtlebot (Color) respond and not any other?

- **Answer:** It's all because of the `ROS_DOMAIN_ID`. In our lab, each team's laptop and robot are set to a different ID number. ROS 2 nodes can only talk to each other if they are on the same ID "channel." So even though we are all shouting to the same topic name `/cmd_vel`, my commands only go to my own robot and won't mess with others.

Q2: Is using the laser scanner a reliable strategy to avoid collisions?

- **Answer:** It's mostly reliable, but **not 100% perfect**.
- **Why it's good:** It's super accurate at seeing things right in front of it on the same flat level, like walls or table legs.
- **Why it can fail:** The laser only scans a flat 2D slice at one specific height. If there is a thin cable on the floor, an overhanging desk corner above its head, or a clear glass door, the laser will completely miss it and the robot will crash right into it.

Q3: What is the detection range of the laser scanner?

- **Answer:** For our TurtleBot3 Burger:
- **Closest it can see:** 12 cm (0.12 m). Anything closer than this is a blind spot and shows up as `0.0` or `inf`.
- **Farthest it can see:** About 3.5 meters (3.5 m).
- **Angle:** A full 360° circle.

Q4: How is the index of the laser scan data mapped to the turtlebot coordinates?

- **Answer:** ROS uses a standard rule where **Forward is the positive X-axis** and **Left is the positive Y-axis**. The laser array starts from the exact front and counts **counter-clockwise**:
- `ranges[0]` : Right in front (0°)
- `ranges[90]` : Directly to the Left (90°)
- `ranges[180]` : Directly behind (180°)
- `ranges[270]` : Directly to the Right (270°)

Q5: What are the other components of the `/cmd_vel` message good for?

- **Answer:** The `/cmd_vel` topic uses a `Twist` message which has 3D linear (x, y, z) and 3D angular (x, y, z) speeds.
- `linear.y` is for drones or robots with special wheels that can slide sideways.
- `linear.z` is for things that go up and down, like drones or submarines.
- `angular.x` and `angular.y` are for tilting or rolling in 3D space.
- Since our TurtleBot is just a simple 2-wheeled ground car, it can only roll forward/backward (`linear.x`) and turn left/right (`angular.z`). The rest are useless for us.

Q6: Explain BEST_EFFORT and KEEP_LAST in the context of QoS.

- **Answer:**
- **BEST_EFFORT** : This means "send the data and hope for the best." If a laser packet gets lost over Wi-Fi, the system doesn't waste time trying to resent it. This is great for fast sensor data because we only care about the newest frame, not old history.
- **KEEP_LAST (with depth=1)**: This means the robot's memory queue only holds the single newest laser image. When a new scan comes in, it throws away the old one. This keeps the robot from lagging and reacting to an old obstacle that it already passed.

Q7: Visualize the timing of your program (rough sketch).

- **Answer:** Here is how the timing flows in my code:

Timeline:	0.0s	0.5s	1.0s	2.0s
Laser Scan	[Path Clear]	[Path Clear]	[OBSTACLE DETECTED]	[Path Clear]
Code Logic	Timer picks speed (Moving Random)	Doing nothing	scan_callback Brakes hard!	Timer picks next Random Speed
Velocity Output	Publishes normal random speed...	Publishes normal random speed...	Publishes Stop Cmd (0, 0.5)	Publishes normal random speed...
Robot Act	Moving normally	Moving normally	STOPS & SPINS	Moves normally again

Q8: What are the weaknesses of your randwalk_safe_turtle.py solution?

- **Answer:**
1. **Side Blind Spots:** My code only checks a 30° window right in front. If the robot is spinning or moving near a wall sideways, it is completely blind and will scrape its sides.
 2. **Stuck in Corners:** The escape plan is dumb—it always spins left (0.5). If it drives into a tight corner where the only exit is to the right, it will just spin left forever and get stuck.
 3. **Bad Braking:** The robot slams on the brakes instantly when something is closer than 30cm. On the real robot, this sudden stop makes the wheels slide, messes up our tracking math (Odometry), and shakes the hardware way too hard.